

CAHIER DES EXIGENCES

Guezzane, Ahmed Fateh
BOIS-DE-BOULOGNE

TABLE DES MATIÈRES

Choix du sujet	2
Base de données.....	2
Affichage de données de la BDD.....	3
Recherche dans la BDD selon plusieurs critères	5
Récupérer un compte selon le nom d'utilisateur.....	5
Récupérer la liste de courses d'un utilisateur selon son ID	6
Récupérer la liste de recette selon la catégorie	7
Récupérer la liste de recette selon un rating	8
Mise à jour de données dans la BDD.....	9
Modification du statut d'un compte.....	9
Suppression d'un compte.....	9
Validation des données.....	10
Validation dans les pages JSP	10
Validation dans les servlets	10
Gestion d'erreurs.....	11
Aspect présentation.....	11
Modèle MVC	12
Pages Web privées (WEB-INF)	12
Séparation des différents types de fichiers.....	13
Nomenclature.....	14
Accès aux données.....	14
Internationalisation	15
Scripts	16
Script de chargement de la vue du tableau de bord	16
Script d'approbation de compte	16
Script de suppression de compte.....	16
Script de création de compte.....	17
Script pour ouvrir les onglets.....	17
Script pour soumettre le formulaire	17

Choix du sujet

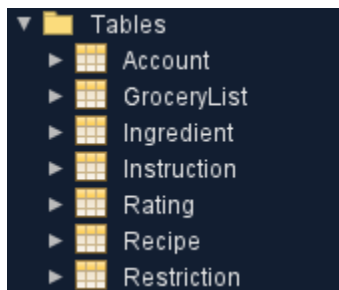
Nous avons décidé de créer une plateforme de recette permettant aux utilisateurs de chercher des recettes par catégorie, diète alimentaire ou de visualiser les recettes avec le meilleur classement. Les membres de la plateforme peuvent rajouter une recette à leur liste de course. Une fois que la recette est dans leur liste de courses, ils peuvent avoir accès à une liste des ingrédients nécessaire pour cuisiner ces recettes. Cette liste d'ingrédient peut être convertie en PDF et puis imprimée par la suite.

Les utilisateurs qui n'ont pas de compte peuvent s'enregistrer. Une fois enregistré, un administrateur doit approuver leur compte afin qu'ils puissent avoir accès aux fonctionnalités de membres.

L'administrateur a accès à un "Dashboard" où il peut approuver, effacer ou rajouter un compte. Éventuellement, il pourra également rajouter, effacer ou modifier une recette mais cette partie n'a pas encore été intégrée.

Base de données

Nous avons une base de données comportant les tables suivantes :

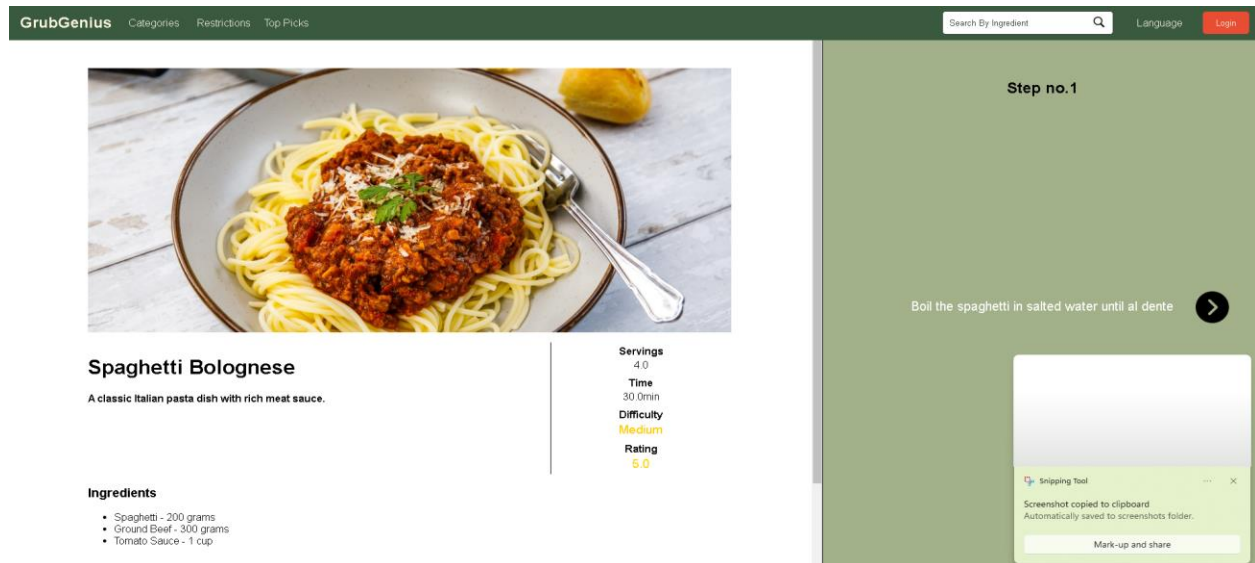


Voici les informations de connexion à cette base de données :

```
private final String DB_HOST = "mysql-ahmedfguezzane.alwaysdata.net:3306";
private final String DB_NAME = "ahmedfguezzane_grubgenius";
private final String DB_USER = "389938_soraya";
private final String DB_PASSWORD = "adminsoraya";
```

Affichage de données de la BDD

Plusieurs méthodes dans nos classes services s'occupent d'appeler les méthodes dans les DAO associé afin de permettre aux servlets d'envoyer les données à afficher aux pages JSP. Voici quelques exemples d'affichage de données :



L'image ci-dessus est le résultat obtenu lorsque l'on sélectionne une recette. Nous sommes renvoyés à la page [recipe.jsp](#) à travers la servlet [RecipeServlet](#). Cette Servlet communique avec un classe Service afin de récupérer les données d'une recette.

```
// If its "openRecipe" (opening a particular recipe)
if("openRecipe".equals(action))
{
    String recipeId = request.getParameter("recipeId");

    Recipe recipe = recService.fetchRecipeById(recipeId); // We fetch that recipe with the id

    // We get the account object from the session
    Account tempAcc = (Account) session.getAttribute("account");

    // We need to know if the user is logged in order to know what to show him on that page
    // For example, the "addToGrocery" button is not available for people that are not logged in
    Boolean isLoggedIn = null;
    isLoggedIn = (Boolean) session.getAttribute("logged");

    // Check if tempAcc is not null and the recipe is in the grocery list
    if (isLoggedIn != null && isLoggedIn) {
        boolean flag = tempAcc.getGroceryList().stream()
            .anyMatch(r -> r.getRecipeID().equals(recipeId)); // We check if that recipe is already in his grocery list
        session.setAttribute("currentRecipeFlag", flag); // If user is logged then we return the flag
    }

    session.setAttribute("currentRecipe", recipe); // We set the currentRecipe attribute to that recipe
    session.setAttribute("instructionCounter", 1); // We also set the InstructionCounter to 1 (for interactive steps)
    response.sendRedirect("recipe.jsp"); // We send him to recipe.jsp and that page will get that recipe with the session Attribute
    return;
}
```

Cette Servlet récupère le ID de la recette que l'on souhaite afficher et appelle la méthode [fetchRecipeById](#) de la classe [RecipeService](#). Une fois cette recette récupérée, nous la plaçons dans

un attribut de session qui sera modifiée à chaque fois qu'une nouvelle recette est sélectionnée. Nous avons également un flag pour savoir si un membre est connecté afin de savoir si nous lui affichons le bouton d'ajout à la liste de courses ou non.

```
// Récupérer une recette par ID
public Recipe fetchRecipeById(String id) {
    return dao.fetchRecipeById(id);
}
```

La classe Service ne fait qu'appeler une méthode de l'interface *IRecipeDAO*. Dans notre cas, nous avons injecté dans notre classe Service *RecipeDAO_DB* qui implémente l'interface *IRecipeDAO*.

Ensuite, notre DAO s'occupe de créer un *PreparedStatement* avec comme argument le ID de la recette que l'on recherche. Cette méthode crée un objet de type *Recipe* et le renvoi au service, qui le renvoi à la servlet et qui a son tour le met dans l'attribut de session mentionné plutôt.

```
@Override
public Recipe fetchRecipeById(String id) {
    String sql = QueryBox.FETCH_RECIPE_BY_ID;
    Recipe recipe = null;
    try {
        PreparedStatement ps = conn.prepareStatement(sql) {
            ps.setString(1, id);
            ResultSet cursor = ps.executeQuery();
            if (cursor.next()) {
                String title = cursor.getString("title");
                String description = cursor.getString("description");
                String category = cursor.getString("category");
                String difficulty = cursor.getString("difficulty");
                double prepTime = cursor.getDouble("prepTime");
                double servings = cursor.getDouble("servings");
                int status = cursor.getInt("status");

                List<Ingredient> ingredientList = fetchIngredientsById(id);
                List<String> instructionList = fetchInstructionsByRecipeId(id);
                List<Restriction> restrictionList = fetchRestrictionsByRecipeId(id);
                List<Rating> ratingList = fetchRatingsByRecipeId(id);

                recipe = new Recipe(id, title, description, ingredientList,
                    restrictionList, category, instructionList,
                    difficulty, prepTime, servings, status, ratingList);
            }
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
    return recipe;
}
```

Voici également la requête SQL qui se trouve dans la classe *QueryBox*. Nous avons mis nos "queries" dans cette classe afin de les centraliser et de faciliter la gestion de celles-ci.

```
public static final String FETCH_RECIPE_BY_ID =
    "SELECT * " +
    "FROM Recipe " +
    "WHERE recipeID = ?";
```

Recherche dans la BDD selon plusieurs critères

Nous avons plusieurs méthodes de “fetching” dans nos DAO qui nous permettent d’afficher les données selon des critères différents. Les classes de services s’occupent d’appeler la méthode dans le DAO lié à l’opération nécessaire.

Afin d’alléger ce document, voici les méthodes dans le DAO qui s’occupent de de rechercher dans la base de données selon plusieurs critères. À noter que toutes les classes (Services, Servlets, DAO, etc.) sont expliquées plus en détail dans le document **RAPPORT COMPLET**.

Récupérer un compte selon le nom d’utilisateur

Cette méthode s’occupe d’accéder à la base de données et de récupérer un compte d’utilisateur avec comme paramètre le nom d’utilisateur. Elle crée un *PreparedStatement*, set le paramètre et crée un objet de type *Account* avec les données liées au compte trouvé dans la base de données.

```
@Override
public Account fetchAccountByUsername(String username) { //VERIFIED
    String sql = QueryBox.FETCH_ACCOUNT_BY_USERNAME;
    Account account = null;
    try {
        PreparedStatement ps = conn.prepareStatement(sql) {

            ps.setString(1, username);
            try (ResultSet cursor = ps.executeQuery()) {
                if (cursor.next()) {
                    String userId = cursor.getString("userId");
                    String firstName = cursor.getString("firstname");
                    String lastName = cursor.getString("lastname");
                    String email = cursor.getString("email");
                    String password = cursor.getString("password");
                    String securityQuestion = cursor.getString("securityQuestion");
                    String securityAnswer = cursor.getString("securityAnswer");
                    int status = Integer.parseInt(cursor.getString("status"));

                    account = new Account(userId, firstName, lastName,
                        username, email, password, securityQuestion,
                        securityAnswer, status, fetchGroceryList(userId));
                }
            }
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
        return account;
    }
}
```

```
public static final String FETCH_ACCOUNT_BY_USERNAME =
    "SELECT * " +
    "FROM Account " +
    "WHERE username = ?";
```

Récupérer la liste de courses d'un utilisateur selon son ID

Cette méthode est très intéressante car elle est un peu plus complexe que les méthodes régulières de ce DAO. Tout d'abord, elle crée un *PreparedStatement* avec comme argument le ID de l'utilisateur et récupère la liste de courses. Ce qui est intéressant c'est que, puisque la liste de course contient des courses qui a leur tour contiennent des ingrédients et des instructions (qui sont dans des tables à part), elle fait appel à un autre DAO (*RecipeDAO*) qui a son tour récupère les données de ces tables grâce a des requêtes différentes et les place dans l'objet recette.

```
@Override
public List<Recipe> fetchGroceryList(String userId) {
    List<Recipe> recipeList = new ArrayList<>();
    // SQL query to get the grocery list for a specific user
    String sql = QueryBox.FETCH_GROCERY_LIST;
    // Step 1: Fetch the recipe IDs for the given userID from the GroceryList table
    try {
        PreparedStatement ps = conn.prepareStatement(sql) {
            ps.setString(1, userId); // Set the userID in the query
            try (ResultSet rs = ps.executeQuery())
            {
                if (rs.next()) {
                    String recipesList = rs.getString("recipesList");
                    // Step 2: Parse the recipesList string into individual recipe IDs
                    String[] recipeIds = recipesList.split(",");
                    // Step 3: Fetch each Recipe by ID
                    for (String recipeId : recipeIds) {
                        // Fetch the recipe using the FETCH_RECIPE_BY_ID query
                        try (PreparedStatement fetchRecipeStmt = conn.prepareStatement(QueryBox.FETCH_RECIPE_BY_ID))
                        {
                            fetchRecipeStmt.setString(1, recipeId); // Set the recipe ID

                            try (ResultSet recipeRs = fetchRecipeStmt.executeQuery()) {
                                if (recipeRs.next()) {
                                    Recipe recipe = new Recipe();
                                    recipe.setRecipeID(recipeRs.getString("recipeID"));
                                    recipe.setTitle(recipeRs.getString("title"));
                                    recipe.setDescription(recipeRs.getString("description"));
                                    recipe.setCategory(recipeRs.getString("category"));
                                    recipe.setDifficulty(recipeRs.getString("difficulty"));
                                    recipe.setPrepTime(recipeRs.getDouble("prepTime"));
                                    recipe.setServings(recipeRs.getDouble("servings"));
                                    recipe.setStatus(recipeRs.getInt("status"));

                                    recipe.setIngredientList(new RecipeDAO_InDB().fetchIngredientsById(recipeId));
                                    recipe.setInstructions(new RecipeDAO_InDB().fetchInstructionsByRecipeId(recipeId));

                                    // Add the recipe to the list
                                    recipeList.add(recipe);
                                }
                            }
                        }
                    }
                }
            }
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
    return recipeList;
}
```

```
public static final String FETCH_GROCERY_LIST = "SELECT recipesList FROM GroceryList WHERE userID = ?";
```

```
public static final String FETCH_INGREDIENTS_BY_RECIPE_ID =
    "SELECT ingredientID, name, quantity, unit " + "FROM Ingredient " + "WHERE recipeID = ?";
```

```
public static final String FETCH_INSTRUCTIONS_BY_RECIPE_ID =
    "SELECT instructions FROM Instruction WHERE recipeID = ?";
```

Récupérer la liste de recette selon la catégorie

Cette méthode de *IRecipeDAO* permet de retourner une liste de recette selon la catégorie passée en argument. À noter que cette méthode ne crée pas un objet Recipe complet avec les ingrédients, etc. Nous avons choisi de le faire avec un constructeur plus léger car cette liste ne sera utilisée que pour afficher la liste de recettes. Une fois que l'utilisateur ou le membre aura sélectionné une recette, nous chargerons la recette entière. De cette façon, l'application Web roulera plus vite.

```
@Override
public List<Recipe> fetchRecipesByCategory(String category) {
    List<Recipe> recipes = new ArrayList<>();

    try (PreparedStatement st = conn.prepareStatement(QueryBox.FETCH_RECIPES_BY_CATEGORY)) {
        st.setString(1, category); // Set the category in the prepared statement
        ResultSet cursor = st.executeQuery(); // Execute the query and get the result set

        while (cursor.next()) {
            String id = cursor.getString("recipeID");
            String title = cursor.getString("title");
            String description = cursor.getString("description");

            Recipe recipe = new Recipe(id, title, description);

            recipes.add(recipe);
        }
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }

    return recipes;
}
```

```
public static final String FETCH_RECIPES_BY_CATEGORY =
    "SELECT r.* FROM Recipe r " +
    "WHERE r.category = ?";
```


Récupérer la liste de recette selon un rating

Cette méthode, à laquelle on passe un double en paramètre, permet de récupérer une liste de recette ayant une note égale ou plus haute que le double passé en paramètre. À noter que pour les mêmes raisons d'optimisation, seul le id, le titre et la description seront chargés dans la recette. Il faut aussi savoir que les Rating de cette recette se trouvent dans une autre table. Chaque rating de chaque utilisateur est une Row de cette table. Donc la méthode doit d'abord calculer la moyenne des notes de toute les "rows" ayant comme *recipeID* le ID de la recette.

```
@Override
public List<Recipe> fetchRecipesWithAvgRatingOver(double ratingThreshold) {

    List<Recipe> recipes = new ArrayList<>();

    try (PreparedStatement st = conn.prepareStatement(QueryBox.FETCH_RECIPES_BY_AVERAGE_RATING)) {
        // Set the rating threshold as the parameter in the prepared statement
        st.setDouble(1, ratingThreshold);
        // Execute the query and get the result set
        ResultSet cursor = st.executeQuery();

        // Iterate through the result set
        while (cursor.next()) {
            String recipeID = cursor.getString("recipeID");
            String title = cursor.getString("title");
            String description = cursor.getString("description");

            Recipe recipe = new Recipe(recipeID, title, description);
            recipes.add(recipe);
        }
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }

    return recipes;
}
```

```
public static final String FETCH_RECIPES_BY_AVERAGE_RATING =
    "SELECT r.recipeID, r.title, r.description, AVG(rt.rate) as avgRating " +
    "FROM Recipe r " +
    "JOIN Rating rt ON r.recipeID = rt.recipeID " +
    "GROUP BY r.recipeID " +
    "HAVING AVG(rt.rate) >= ?";
```

Mise à jour de données dans la BDD

Modification du statut d'un compte

Cette méthode permet à l'administrateur, à travers son tableau de bord, d'approuver le compte d'un membre. Elle accède à la base de données et met comme valeur 1 à la colonne statut. Il faut noter que cette méthode retourne un *int* indiquant le nombre de "row" qui ont été affecté afin de permettre à la page JSP (à travers la servlet et la classe service) d'afficher un message de succès ou d'erreur.

```
@Override
public int approveAccount(String userId) {
    String sql = QueryBox.APPROVE_ACCOUNT;
    try {
        PreparedStatement ps = conn.prepareStatement(sql) {
            ps.setString(1, userId);
            return ps.executeUpdate();
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}
```

```
public static final String APPROVE_ACCOUNT =
    "UPDATE Account " +
    "SET status = 1 " +
    "WHERE userID = ?";
```

Suppression d'un compte

Cette méthode permet à l'administrateur, à travers son tableau de bord, de supprimer un compte de la base de données. Il faut noter que cette méthode retourne un int indiquant le nombre de row qui ont été affecté afin de permettre à la page JSP (à travers la servlet et la classe service) d'afficher un message de succès ou d'erreur.

```
//DELETE
@Override
public int deleteAccount(String userId) {
    String sql = QueryBox.DELETE_ACCOUNT;

    try {
        PreparedStatement ps = conn.prepareStatement(sql) {
            ps.setString(1, userId);

            int rowsAffected = ps.executeUpdate(); // Returns the number of rows affected
            return rowsAffected; // Return the result directly
        } catch (SQLException e) {
            throw new RuntimeException("Error while deleting account: " + e.getMessage(), e);
        }
    }
}
```

```
public static final String DELETE_ACCOUNT =
    "DELETE FROM Account " +
    "WHERE userID = ?";
```

Validation des données

La validation des données est faite à plusieurs niveau. Nous validons les données directement depuis les pages JSP dans les formulaires mais également au niveau des Servlets.

Validation dans les pages JSP

Les types d'input dans les formulaires sont paramétrés de façon que l'utilisateur reçoive une alerte si le mauvais type d'information est entré. Comme par exemple, si l'utilisateur ne rentre pas une adresse courriel dans le champ 'email', le formulaire lui enverra une alerte afin qu'il fasse la correction.

```
<form class="register-form" action="AccountServlet" method = "POST">
  <input type="text" name="username" placeholder="<fmt:message key="register.username"/>" required>
  <input type="text" name="firstName" placeholder="<fmt:message key="register.firstName"/>" required>
  <input type="text" name="lastName" placeholder="<fmt:message key="register.lastName"/>" required>
  <input type="email" name="email" placeholder="<fmt:message key="register.email"/>" required>
  <input type="password" name="password" placeholder="<fmt:message key="register.password"/>" required>
  <input type="password" name="confirmPassword" placeholder="<fmt:message key="register.confirm"/>" required>
  <input type="text" name="securityQuestion" placeholder="<fmt:message key="register.securityQ"/>" required>
  <input type="text" name="securityAnswer" placeholder="<fmt:message key="register.securityA"/>" required>
  <input type="hidden" name="accountAction" value="register"/>
  <button type="submit"><fmt:message key="register.button"/></button>
</form>
```

Validation dans les servlets

Nous validons également les données dans les servlets. Comme dans la servlet *AccountServlet*, ou l'enregistrement d'un compte se fait, nous validons que le nom d'utilisateur choisi est disponible mais également que le mot de passe et la confirmation de mot de passe sont exactement similaire.

```
// If its register (on register page)
if("register".equals(accountAction))
{
    session.setAttribute("registerSuccess", false);
    session.setAttribute("registerResponse", 5);
    // We collect the information
    String username = request.getParameter("username");
    String firstName = request.getParameter("firstName");
    String lastName = request.getParameter("lastName");
    String email = request.getParameter("email");
    String password = request.getParameter("password");
    String confirmPassword = request.getParameter("confirmPassword");
    String securityQuestion = request.getParameter("securityQuestion");
    String securityAnswer = request.getParameter("securityAnswer");

    // We make sur the password and confirmPassword match
    if(!password.equals(confirmPassword))
    {
        session.setAttribute("registerResponse", -2);
        response.sendRedirect("register.jsp"); // We return the code -2 which means password dont match
        return;
    }
    // Call accService.submitAccount once and check the result
    int result = accService.submitAccount(username, firstName, lastName, email, password, securityQuestion, securityAnswer, 0);

    if (result == 0 || result == -1) {
        session.setAttribute("registerResponse", result);
        // Handle failure (either result == 0 which means it didnt work or result == -1 because the username is already used)
        response.sendRedirect("register.jsp");
    }
    else
    {
        // We redirect to login this time and the success code is one which means he registered correctly
        session.setAttribute("registerSuccess", true);
        response.sendRedirect("login.jsp");
    }
}
}
```

Gestion d'erreurs

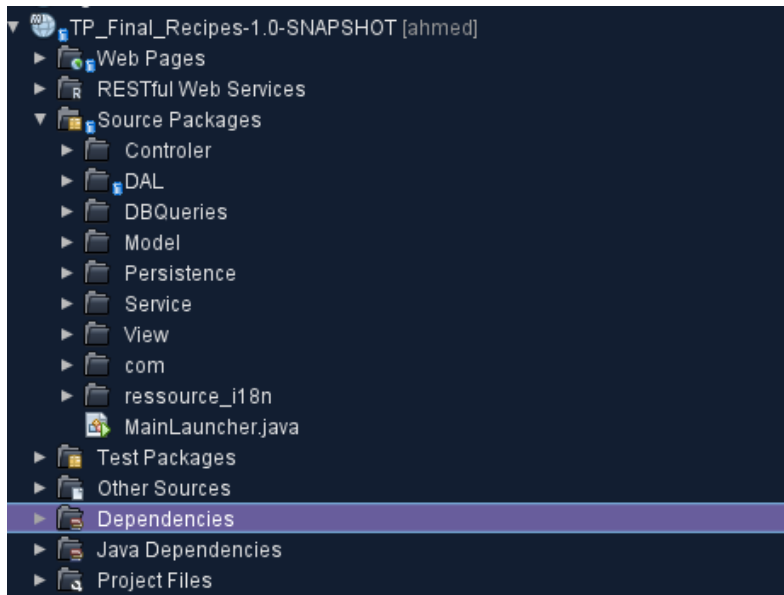
Dans l'ensemble de notre code au sein des, qui sont responsables de la communication avec la base de données, chaque instruction potentiellement risquée est encapsulée dans des blocs "Try-and-Catch". Cette pratique permet de détecter et de gérer les exceptions qui pourraient survenir pendant l'exécution du programme (runtime), comme des erreurs de connexion à la base de données, des requêtes SQL mal formulées, ou des problèmes liés à la manipulation des données. En utilisant ces blocs, nous nous assurons que l'application reste stable et fonctionnelle, même en cas d'erreur.

Aspect présentation

Notre application web a été conçue en plaçant l'expérience utilisateur et l'esthétique au cœur du développement. Chaque page a été soigneusement travaillée en utilisant des feuilles de style CSS pour garantir une interface à la fois moderne, cohérente et agréable à utiliser. Nous avons accordé une attention particulière aux détails visuels, tels que les espacements, les polices de caractères, les animations subtiles et les transitions fluides, afin d'offrir une navigation intuitive et immersive. De plus, les palettes de couleurs ont été méticuleusement sélectionnées pour créer une harmonie visuelle, refléter l'identité de l'application et renforcer l'engagement des utilisateurs. Ce souci du design vise non seulement à rendre l'application plus attrayante, mais aussi à améliorer l'accessibilité et le confort d'utilisation pour tous les utilisateurs.

Le document **GUIDE INTERFACE GRAPHIQUE** fait le tour de toutes les pages afin d'expliquer les composants des pages. Il donne un très bel aperçu du rendement visuel de l'Application Web.

Modèle MVC



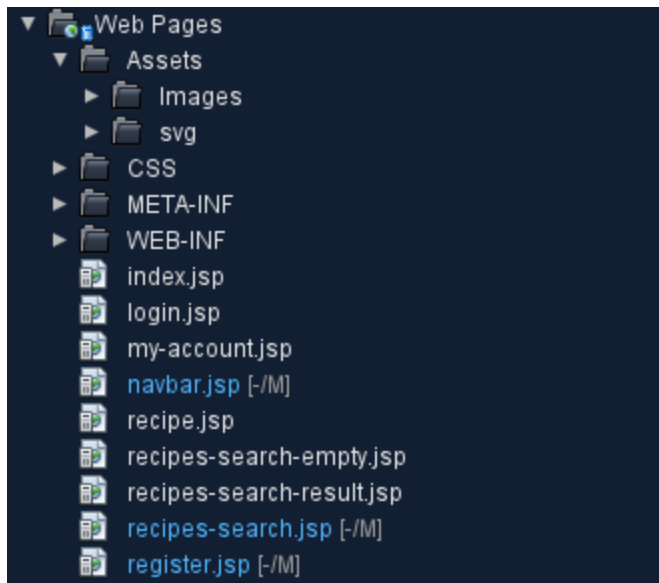
Notre application web a été structurée en suivant rigoureusement le modèle MVC (Modèle-Vue-Contrôleur), une architecture logicielle qui permet de séparer clairement les responsabilités entre les différentes couches de l'application. Le **Modèle** gère la logique métier et les interactions avec la base de données, garantissant ainsi une gestion efficace et sécurisée des données. La **Vue** est dédiée à la présentation et à l'interface.

Enfin, le **Contrôleur** joue le rôle d'intermédiaire entre le Modèle et la Vue, traitant les requêtes de l'utilisateur, orchestrant les actions nécessaires et renvoyant les réponses appropriées. Cette séparation des préoccupations facilite la maintenance, la scalabilité et la collaboration entre les développeurs, tout en assurant une base de code propre et organisée.

Pages Web privées (WEB-INF)

Dans notre application web, les pages liées au tableau de bord administratif ont été placées dans le dossier **WEB-INF**, une pratique courante pour renforcer la sécurité. En effet, ce dossier est inaccessible directement par les utilisateurs via l'URL, ce qui empêche tout accès non autorisé à ces pages sensibles. Seuls les contrôleurs de l'application, après vérification des permissions, peuvent rediriger vers ces pages, garantissant ainsi que seuls les administrateurs authentifiés y ont accès. Cependant, nous reconnaissons une lacune dans cette approche : la page [my-account.jsp](#), bien qu'elle contienne des informations sensibles liées aux comptes utilisateurs, n'a pas été placée dans le dossier WEB-INF. Cette omission représente un risque potentiel pour la sécurité, car cette page pourrait être exposée à des accès non autorisés. Il serait donc judicieux de la déplacer dans le dossier WEB-INF pour aligner sa gestion avec les bonnes pratiques de sécurité déjà mises en œuvre pour les autres pages administratives.

Séparation des différents types de fichiers



Dans notre application web, nous avons organisé les fichiers de manière structurée pour faciliter la maintenance et améliorer la clarté du projet. Les fichiers statiques, tels que les images, les feuilles de style CSS et les scripts JavaScript, ont été placés dans des dossiers distincts (par exemple, /images, /css) afin de séparer les ressources selon leur type et de simplifier leur gestion. Cette organisation permet une meilleure lisibilité du code et une optimisation des chemins d'accès. En ce qui concerne les fichiers JSP, ils ont été placés directement dans le dossier **Web Pages**, ce

qui fonctionne mais n'est pas idéal d'un point de vue organisationnel. Il aurait été plus judicieux de les regrouper dans un dossier dédié, par exemple /jsp, pour centraliser les fichiers dynamiques et renforcer la cohérence de l'architecture. Cette approche aurait également permis de mieux distinguer les fichiers JSP des autres ressources et de simplifier les contrôles d'accès ou les modifications futures.

Nomenclature

Dans notre application web, les noms des classes ont été choisis de manière significative afin de faciliter la compréhension et la navigation dans le code. Chaque classe porte un nom descriptif qui reflète clairement son rôle ou sa fonction, permettant ainsi aux développeurs de s'y retrouver rapidement. Par exemple, les classes modèles comme `User` ou `Product` indiquent directement les entités qu'elles représentent. En revanche, pour les **DAO** et les **classes de service**, une nomenclature spécifique a été respectée : les DAO sont systématiquement suffixés par `DAO` (comme `AccountDAO` ou `RecipeDAO`), et les classes de service par `Service` (comme `AccountService` ou `RecipeService`). Cette approche mixte, combinant des noms significatifs pour la plupart des classes et une nomenclature stricte pour les DAO et les services, contribue à une base de code à la fois lisible, organisée et facile à maintenir.

Accès aux données

Lors de la configuration de la base de données pour notre application, nous avons rencontré des difficultés techniques pour intégrer MySQL. Afin de ne pas perdre de temps et de pouvoir avancer efficacement sur le projet, nous avons opté pour `MariaDB`. Cette décision nous a permis de poursuivre le développement sans interruption. Le driver `.jar` de `MariaDB` a été correctement placé dans le dossier `lib` du projet, conformément aux bonnes pratiques, et la configuration de la connexion à la base de données a été testée avec succès. Ainsi, tout est en place pour que la base de données soit pleinement fonctionnelle et que l'application puisse interagir avec elle de manière fluide et sécurisée.

Internationalisation

Nous avons internationalisé (i18n) notre application web pour la rendre accessible à un public anglophone et francophone. Toutes les informations textuelles de l'interface utilisateur (qui ne proviennent pas de la base de données) ont été traduites en français et en anglais. L'utilisateur peut choisir sa langue préférée directement depuis n'importe quelle page de l'application grâce à un sélecteur de langue. Les traductions sont gérées via deux classes Java, *Messages_fr* et *Messages*, qui étendent *ListResourceBundle* et sont situées dans le dossier ressource_i18n. Ces classes contiennent les paires clé-valeur pour chaque texte traduit. Pour gérer le changement de langue, nous avons implémenté un servlet appelé *GetLocalRequest*, qui est invoqué lorsque l'utilisateur sélectionne une nouvelle langue. Ce servlet met à jour la locale de l'utilisateur et redirige vers la page actuelle avec la langue appropriée. Enfin, pour afficher les textes traduits dans les pages JSP, nous avons intégré la balise

`<%@ taglib uri="http://java.sun.com/jsp/jstl/fmt" prefix="fmt" %>`

dans chaque page, permettant d'utiliser la bibliothèque JSTL (JavaServer Pages Standard Tag Library) pour formater et afficher dynamiquement les contenus en fonction de la locale sélectionnée.

```
protected void processRequest(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    HttpSession session = request.getSession();

    String userLocale = request.getParameter("userlocale"); // We get the language selection from the language dropdown
    String referer = request.getHeader("Referer"); // We also get the page he was in

    // If for some reason the parameter returns null, by default the language is english
    if (userLocale == null) {
        userLocale = "en";
    }
    session.setAttribute("locale", userLocale); // We set a session attribute with the selection

    // If the referer is not null, we send him back to where he comes from
    if (referer != null) {
        response.sendRedirect(referer);
    } else {
        response.sendRedirect("index.jsp"); // Or we send him back to index.jsp
    }
}
```

```
<fmt:setLocale value = "${sessionScope.locale}"/>
<fmt:setBundle basename = "ressource_i18n.Messages"/>
```

Si aucune valeur n'est trouvée dans le `sessionScope.locale`, il prend directement la traduction dans `Messages.java`

Scripts

Script de chargement de la vue du tableau de bord

** Se trouve dans [Web Pages/WEB-INF/jsp/adminPanel.jsp](#)

Ce script permet de rendre dynamique le tableau de bord en écoutant les clics sur les sections de la barre latérale ([.panel-section](#)). Lorsqu'une section est cliquée, il récupère l'URL associée via l'attribut data-url, affiche un "skeleton loader" pendant le chargement, puis utilise l'API [fetch](#) pour charger et injecter le contenu dans l'élément [dashboard-view](#). En cas de succès, le contenu est affiché ; en cas d'erreur, un message d'erreur est affiché. Le script gère également l'état actif des sections en ajoutant ou supprimant la classe active, offrant ainsi une navigation fluide et réactive.

Script d'approbation de compte

** Fonction : [approveAccount](#)

** Se trouve dans [Web Pages/WEB-INF/jsp/adminPanel.jsp](#)

Ce script permet d'approuver un compte utilisateur en envoyant une requête POST au serveur avec le [userId](#) et une action spécifique ([approveUserAccount](#)). Si la requête réussit, il met à jour le contenu de la page "approbation des comptes" dans la vue du tableau de bord en rechargeant dynamiquement le contenu via une seconde requête. En cas d'erreur, une alerte affiche un message d'erreur pour informer l'utilisateur. Le script assure ainsi une gestion fluide et réactive des approbations de comptes.

Script de suppression de compte

** Fonction : [deleteAccount](#)

** Se trouve dans [Web Pages/WEB-INF/jsp/adminPanel.jsp](#)

Ce script permet de supprimer un compte utilisateur en envoyant une requête POST au serveur avec le [userId](#) et une action spécifique ([deleteUserAccount](#)). Si la suppression réussit, il met à jour dynamiquement la page "suppression des comptes" dans la vue du tableau de bord en rechargeant son contenu via une seconde requête. En cas d'erreur, une alerte affiche un message d'erreur pour informer l'utilisateur. Le script assure ainsi une gestion fluide et réactive des suppressions de comptes, tout en maintenant l'interface à jour.

Script de création de compte

** Fonction: *createAccount*

** Se trouve dans *Web Pages/WEB-INF/jsp/adminPanel.jsp*

Ce script permet de créer un nouveau compte utilisateur en collectant les données du formulaire ayant l'ID *createAccountForm*, en y ajoutant une action spécifique (*createUserAccount*), et en envoyant une requête POST au serveur. Si la création réussit, il recharge dynamiquement la section "ajouter des comptes" dans la vue du tableau de bord pour afficher les mises à jour. En cas d'erreur, une alerte affiche un message d'erreur pour informer l'utilisateur. Le script assure ainsi une gestion fluide et réactive de la création de comptes, tout en maintenant l'interface à jour.

Script pour ouvrir les onglets

** Fonction : *toggleSubCategory*

** Se trouve dans *Web Pages/recipes-search.jsp*

Ce script permet de basculer l'affichage des sous-catégories (comme "Régime", "Viande", "Légumes", etc.) lorsque l'utilisateur clique sur le titre de la sous-catégorie. Il ajoute ou retire la classe open pour ouvrir ou fermer la sous-catégorie, tout en changeant dynamiquement le symbole du titre : un symbole moins (−) indique que la sous-catégorie est ouverte, tandis qu'un symbole plus (+) indique qu'elle est fermée. Cette fonctionnalité améliore l'interactivité et la navigation dans les catégories.

Script pour soumettre le formulaire

** Fonction : *submitCategoryForm*

** Se trouve dans *Web Pages/recipes-search.jsp*

Ce script permet de soumettre un formulaire de recherche de recettes en fonction des filtres sélectionnés (restriction alimentaire, ingrédient ou type de recette). Il définit les valeurs des champs cachés du formulaire en fonction des paramètres fournis, puis soumet le formulaire. Cette fonctionnalité permet de déclencher une recherche dynamique en fonction des filtres choisis, offrant une expérience utilisateur fluide et réactive.